

第 8 章 开发实例解析

8.1 实例描述

本章将在前面章节基础上，给出一个自动生产售货机的例子，目的是通过一个可运行的例子来体现如何将设计原则、设计模式应用到具体应用中，毕竟只有将设计原则、设计模式应用到具体系统设计开发中，才能体现其完整价值。在例子中，具体应用了装饰器模式、抽象工厂模式以及 MVC 模式，其中 MVC 模式是一种架构模式，也是集多种模式的综合性模式，在实际项目中使用尤其广泛。这三种模式的引入是对第六章内容的扩展与深化，希望可以扩大读者的知识面。

首先来介绍下自动生产售货机系统（以下简称售货机系统）。假设一个零售商店出售饮料，店主想建立这样一个售货机系统，售货机系统能提供两种饮料，分别是 coca 和 Coffee。当客户提出请求后，售货机系统就能够按用户要求“制造”出饮料，然后将价格打印出来。为了满足客户口味的多样性，售货机系统也可以在用户点的饮料里增加一些配料，配料有 milk 和 ice；当然这些配料也是要钱的，打印单上也要增加上相应的价钱。用户向售货机系统提交请求，售货机系统将用户需求列出来，并显示价格；如果用户请求出错，系统将给出提示信息。

8.2 售货机系统的设计和实现

8.2.1 各式饮料的实现

如何实现各式的饮料呢，比如加 milk 的 coca 或者加 ice 的 coffee？从面向对象的角度考虑，每种饮料和配料都应该可以成为一个类，用户购买各式的、不同口味的饮料，相当于给一个对象增加了一些额外的职责。创建子类的方式是可以比较容易想到的：比如创建 milk_coca 类，那么就可以直接用这个类来代表饮料加 milk 的 coca 了。但是让我们仔细地考虑这种方法是否合适。如果售货机系统增加了饮料数目以及配料数目，现在有了 10 种饮料和 10 种配料，那么按照创建子类的方法来实现各种不同口味的饮料的话需要新创建多少个新类呢？ $10 \times 10 = 100$ 个！不仅如此，如果商店里再增加饮料或者配料呢？或者在一份饮料里加入多份配料呢（比如 coca 里加两份 milk）？

经过上面的分析，显然可以看出通过增加子类来实现为一个对象增加功能是不够灵活的。实际上，在创建新的类的时候，就是反复地使用原有的类，把各种饮料和配料组合起来。以饮料为对象，各种配料的加入就相当于增加了原有饮料的一些附加功能。对这样的情况，有什么模式可以利用和借鉴？使用装饰器模式。装饰器模式的意图是可以动态地给一个对象添加一些额外的职责，就增加功能而言，装饰模式比生成子类更为灵活。装饰器模式的结构如下图 8-1 所示，Component 是定义一个对象接口，可以给这些对象动态地添加职责。

ConcreteComponent 是定义了一个具体的对象，也可以给这个对象添加一些职责。Decorator，装饰抽象类，继承了 Component，从外类来扩展 Component 类的功能，但对于 Component 来说，是无需知道 Decorator 的存在的。至于 ConcreteComponent 就是具体的装饰对象，起到给 Component 添加职责的功能。不难理解，面对变化，如果采用生成子类的方法进行扩充，为支持每一种扩展的组合，会产生大量的子类，使得子类数目呈爆炸性增长。装饰器模式可以更加灵活地以动态、透明的方式给单个对象增加职责。

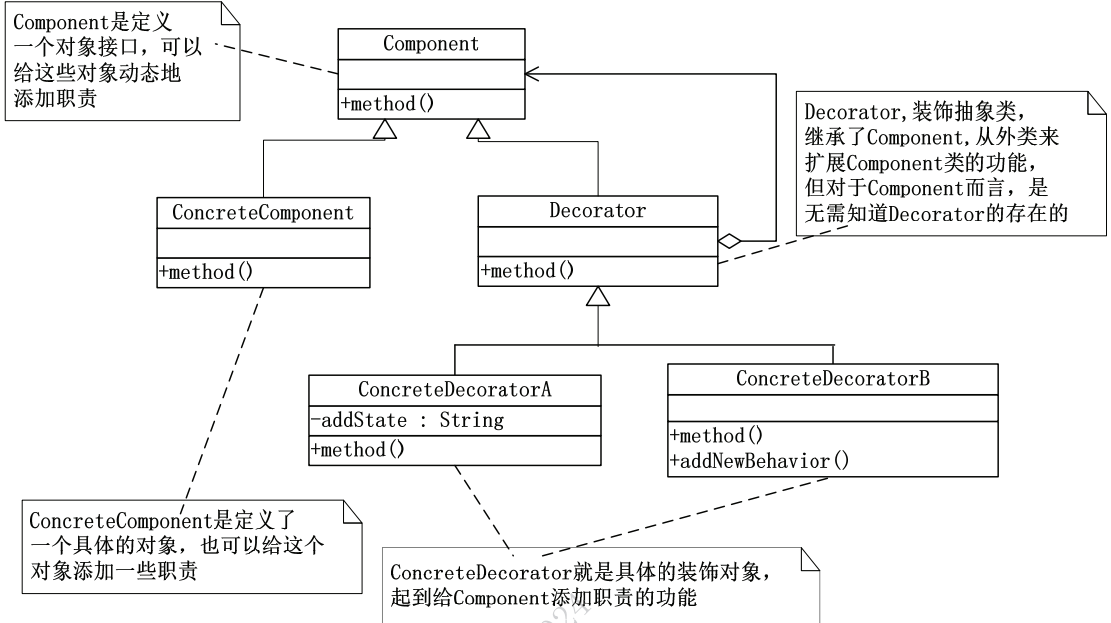


图 8-1 装饰器模式结构图

借鉴装饰器模式，设计现有的售货机系统，系统的类图如下图 8-2 所示：

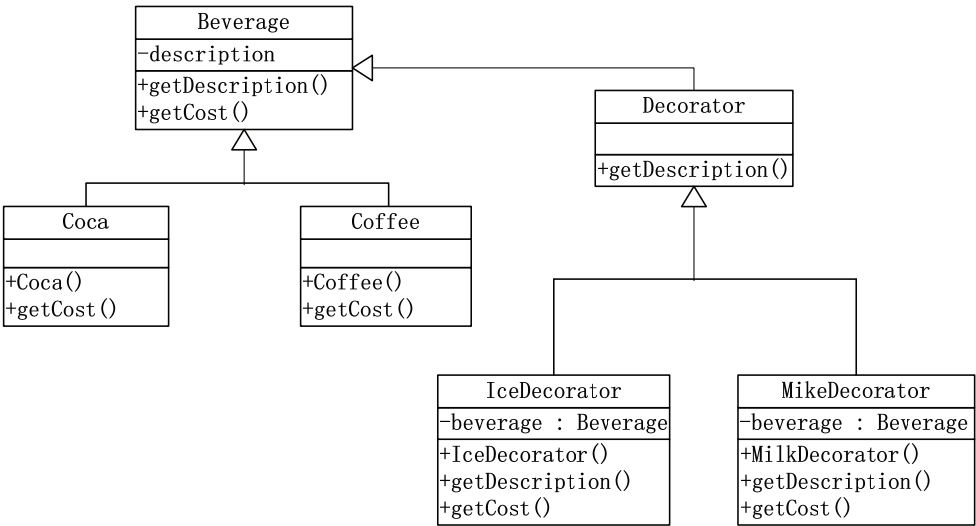


图 8-2 售货机系统的类图

这样，配料对象对饮料对象进行“装饰”，自动售货机系统就可以生产各式口味的饮料了；而且，因为 Decorator 类继承于 Beverage 类，故饮料可以被配料装饰多次，因此加入多份配料也就变得灵活很多。比如一个用户类 client 现在来购买一份加 milk 的 coca 饮料，客

户端可以在自己类里面先创建一个 coca 对象（Beverage beverage = new Coca（）），然后再用 milk 装饰一下（beverage = new MilkDecorator（beverage））即可。

下面用 Java 语言具体实现一下：

(1) Beverage 是一切饮料的父类，是抽象类

```
/*
 *Beverage.java
 */
package lhl.hhu;

/*
 * 装饰者模式---动态地将职责附加到对象上。若要扩展功能，装饰者提供了比继承更有弹性的替代方案。
 这是所有饮料的父类，在这个例子中，我们实现了两种饮料：Coca 和 Coffee，
 它们都继承于此类。
```

```
*/
public abstract class Beverage {
    String description = "Unknown Beverage.";
    //decription 用来描述当前的饮料
    public String getDescription(){
        return description;
    }
    //此方法用来计算饮料的费用
    public abstract double getCost();
}
2024
```

(2) coca 和 Coffee 是具体的饮料，它们直接继承于父类 Beverage

```
/*
 *Coca.java
 */
package lhl.hhu;
public class Coca extends Beverage {
    //这是 Coca 本身的描述和价格
    public Coca(){
        description = "Coca Beverage";
    }
    public double getCost() {
        return 1;
    }
}
/*
 *Coffee.java
 */
package lhl.hhu;
public class Coffee extends Beverage {
```

```

//这里是 Coffee 本身的描述和价格
public Coffee(){
    description = "Coffee Beverage";
}
public double getCost() {
    return 2;
}
}

```

(3) Decorator 是装饰者模式父类，为所有配料类继承，且本身继承于 Beverage 类。

```

/*
 *Decorator.java
 */
package lhl.hhu;

/*
 * 这个抽象类是用来作为所有装饰者的父类，为了使其能够装饰饮料类，
 * 它继承了与饮料(Coca 和 Coffee)相同的父类 Beverage，也因此，
 * 对于 Beverage 来说是可以被装饰多次的。
 */

public abstract class Decorator extends Beverage {
    public abstract String getDescription();
}

```

(4) MilkDecorator 和 IceDecorator 是具体的装饰类

```

/*
 *MilkDecorator.java
 */
package lhl.hhu;

public class MilkDecorator extends Decorator {

    /*
     * MilkDecorator 的构造方法里传入需要装饰的饮料(反过来看就是把 milk 加到饮料中
     去)，这样就可以引用被装饰的饮料的描述和价格了。
     */
    Beverage beverage;

    public MilkDecorator(Beverage beverage){
        this.beverage = beverage;
    }

    /* 这里返回饮料的信息和装饰者本身的信息，

```

```

        Milk 是该装饰者(MilkDecorator)的描述，0.5 是价格。
    */
    public String getDescription() {
        return beverage.getDescription()+" 和 Milk";
    }

    public double getCost() {
        return beverage.getCost()+0.5;
    }
}

/*
 *IceDecorator.java
 */
package lhl.hhu;

public class IceDecorator extends Decorator {
    /*
     * 同样 IceDecorator 的构造方法里传入需要装饰的饮料(反过来看就是把 ice 加到饮料
    中去)
     * 这样就可以引用被装饰的饮料的描述和价格。
     */

    Beverage beverage;
    public IceDecorator(Beverage beverage){
        this.beverage = beverage;
    }

    /* 这里返回饮料的信息和装饰者本身的信息，
     Ice 是该装饰者(IceDecorator)的描述，0.5 是价格。
     */
    public String getDescription() {
        return beverage.getDescription()+" 和 Ice";
    }

    public double getCost() {
        return beverage.getCost()+0.5;
    }
}

```

8.2.2 工厂模式的引入

现在，可以进一步考虑如何满足不同用户的购买需求。因为，购买的饮料和装饰都是直接编码在客户端中的，它的购买的对象也就没办法改变了（除非修改源代码）。另外，如果

商店里增加了一种新的饮料或是配料，用户又怎么选择购买这些商品呢？

结合第六章的内容，很容易地想到，此时可以考虑使用工厂模式。用户需要什么，就向工厂提出请求，由工厂来“生产”各种不同的口味配料的饮料给用户，而用户不需要关心自己所要求的商品是如何造出来的。

下图 8-3 是引入工厂模式后设计的部分类图：

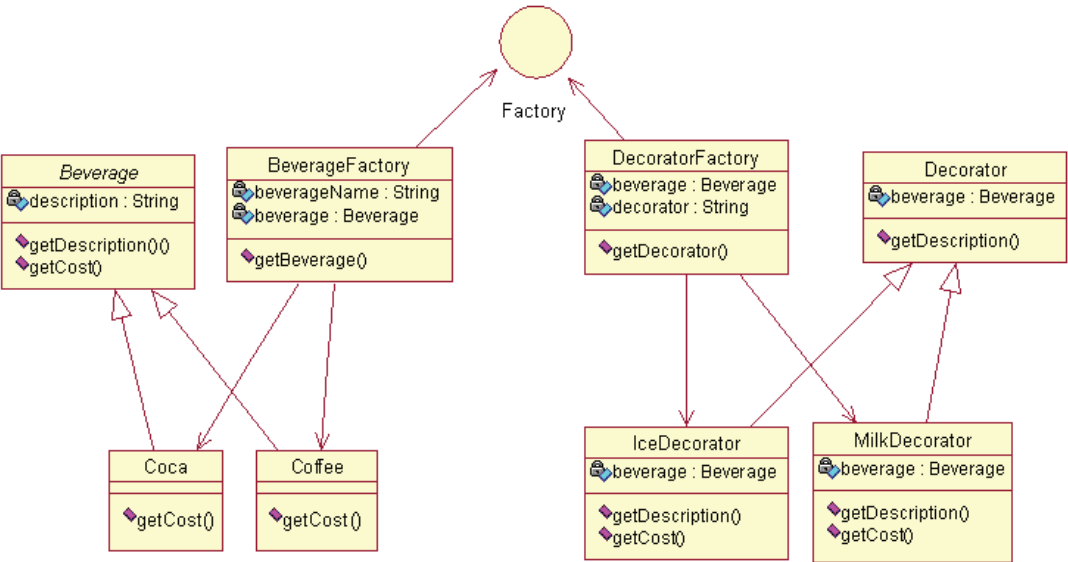


图 8-3 引入工厂模式后系统的部分类图

(1) **factory** 是一切工厂类的父类，是接口

```
/*
 * Factory.java
 */
package lhl.hhu;
/* 这是个接口，所有工厂均实现这个接口，在这个例子中这个接口
 * 只是提供了统一的外观。
 */
public interface Factory {
}
```

(2) **BeverageFactory** 类是饮料工厂，负责生产饮料

```
/*
 * BeverageFactory.java
 */
package lhl.hhu;

/*这里是饮料的生产地，所有的 beverage 实例都是在此产生。针对不同的情况(饮料名的不同)生产不同的产品。当有新饮料需要加入的时候，只需要加入新产品的代码（如 Tea，要继承 Beverage 类），然后在此加上对应的“生产”代码就行了，不需要修改其他地方的代码。
 */
```

```

public class BeverageFactory implements Factory{

    private String beverageName;
    private Beverage beverage;

    public Beverage getBeverage(String beverageName){
        this.beverageName = beverageName;
        if(beverageName.equals("coca"))
            beverage = new Coca();
        else if(beverageName.equals("coffee"))
            beverage = new Coffee();

        //这里的 NoBeverage 对象可以处理各种其他情况，具体实现见“非法请求处理”
        else beverage = new NoBeverage();

        return beverage;
    }
}

```

(3) DecoratorFactory 是装饰者工厂，负责生产配料。

```

/*
 *DecoratorFactory.java
 */
package lhl.hhu;

/*
    这里是配料的生产地，所有的 Decorator 实例都是在此产生。针对不同的情况（配料名
    的不同）生产不同的产品。当有新配料需要加入的时候，只需要加入新产品的代码（如 Suger，
    要继承 Decorator 类），然后在此加上对应的代码就行了，不需要修改其他地方的代码。
 */
public class DecoratorFactory implements Factory {
    private String decorator;
    private Beverage beverage;
    public Beverage getDecorator(String decorator,Beverage beverage){
        this.decorator = decorator;
        this.beverage = beverage;
        if(decorator.equals("milk"))
            beverage = new MilkDecorator(beverage);
        else if(decorator.equals("ice"))
            beverage = new IceDecorator(beverage);

        //这里的 NoDecorator 对象可以处理各种其他情况，具体实现见“非法请求处理”
        else beverage = new NoDecorator();
        return beverage;
    }
}

```

}

8.2.3 用户购买的请求与反馈

对于用户向售货机系统提交购买饮料的请求以及系统反馈价格信息给用户，我们采用 MVC 模式。MVC 包括三类对象，Model 即模型、视图、控制器，它以会话方式提供方便的 GUI 支持，从面向对象的角度看，这种结构可以使程序具有很好的对象化特性。其中，模型用于存储数据的对象；视图向控制器提交所需数据、显示模型中的数据；控制器负责具体的业务逻辑操作，即根据视图提出的要求对数据进行处理，将有关结果存储到模型中。如果不使用 MVC，用户界面设计往往将业务逻辑、界面表示等混在一起，而 MVC 则将它们分离以提高灵活性和复用性。

在 web 服务的技術中，JSP 页面擅长数据的显示，适合于作为用户的视图，我们可以采取一种 JSP+Servlet+JavaBean 来构建 MVC；这也是目前 web 服务中常用的结构。其中 JavaBean 仅仅负责提供修改和返回数据的方法，Servlet 负责处理对象，并将结果存储到 JavaBean 对象中，实现了各个部分的松散耦合，也更加便于维护。

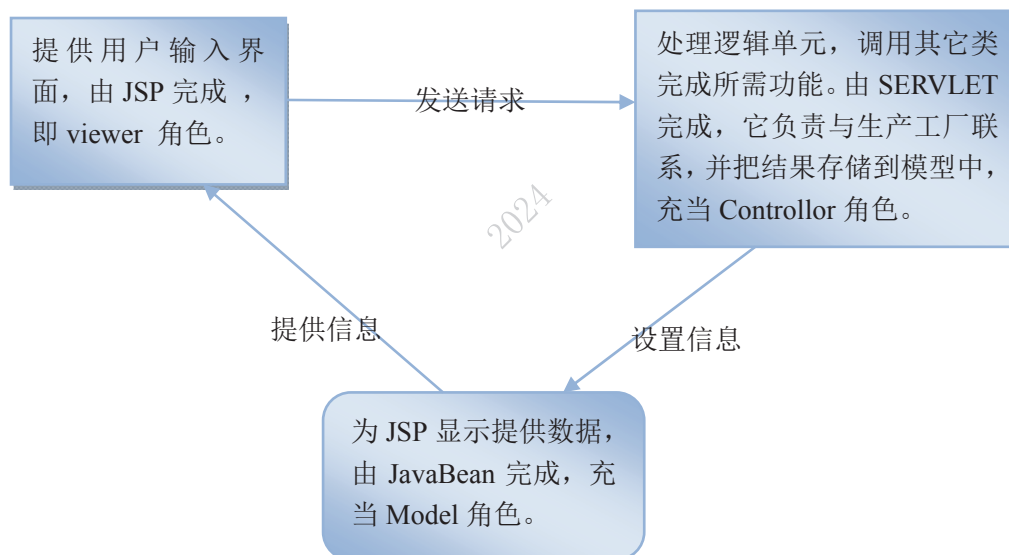


图 8-4 Web 服务中的 MVC 模式

设计代码：

(1) index.jsp

```
<%@ page contentType="text/html;Charset=gb2312" pageEncoding="gb2312"%>
<%@ page import="lhl.hhu.ShopClientBean"%>
<jsp: useBean id="client" class="lhl.hhu.ShopClientBean" scope="request"/>
<html>
  <head>
    <title>shop</title>
    <script>
```



```

        function isDigit(s)
        {
            //这个函数用来判定输入的信息必须为数字，使用 JavaScript 编写。
            var patrn=/^[0-9]{1,20}$/;
            if (!patrn.exec(s)) alert("这里必须输入数字！ ");
        }
    </script>
</head>

<body>
    <form action="shopService" method="post">
        您需要点的饮料是： <input type="text" name="product" value="<jsp: getProperty
name="client" property="product"/>"><br/>
        您需要给饮料加的配料是： <input type="text" name="decorator" value="<jsp: getProperty
name="client" property="decorator"/>"><br/>
        配料可以加多份(不填默认为 1 份)： <input type="text" name="num" value="1"
onfocusout="isDigit(this.value)"><br/>
        <input type="submit" value="提交"><input type="reset" value="重置"><br/>
        您所点的饮料和配料是： <jsp: getProperty name="client" property="description"/><br/>
        您所点的所有饮料价格是： <jsp: getProperty name="client" property="price"/>
    </form>
</body>
</html>

```

(2) ShopClientBean 类，完成 JavaBean 功能。

```

/*
 *ShopClientBean.java
 */
package lhl.hhu;

public class ShopClientBean {
    private String product = "";
    private String decorator = "";
    private String description = "您还没有点饮料。";
    private double price = 0;

    public String getProduct() {
        return product;
    }
}

```

```

    public void setProduct(String product) {
        this.product = product;
    }
    public String getDecorator() {
        return decorator;
    }
    public void setDecorator(String decorator) {
        this.decorator = decorator;
    }
    public double getPrice() {
        return price;
    }
    public void setPrice(double price) {
        this.price = price;
    }
    public String getDescription() {
        return description;
    }
    public void setDescription(String description) {
        this.description = description;
    }
}

```

(3) ShopService 类，完成 Servlet 功能，处理单元

/*

***ShopService.java**

*/

```
package lhl.hhu;
```

//下面主要是导入网络服务所需要的一些类和异常类。

```
import java.io.IOException;
```

```
import javax.servlet.RequestDispatcher;
```

```
import javax.servlet.ServletException;
```

```
import javax.servlet.http.HttpServlet;
```

```
import javax.servlet.http.HttpServletRequest;
```

```
import javax.servlet.http.HttpServletResponse;
```

```
public class ShopService extends HttpServlet{
```

```
public void doPost(HttpServletRequest request, HttpServletResponse response)
    throws ServletException, IOException {
```

```
//Servlet 要继承 HttpServlet 类，然后才能实现对用户的服务功能。
```

```
//下面是参数定义
```

```
String product;
```

```
String decorator;
```

```
String num;
```

```
String description="";
```

```
double price;
```

```
int number = 1;
```

```
//建立模型对象 bean，并把它置到请求头对象 request 中去
```

```
ShopClientBean bean = new ShopClientBean();
```

```
request.setAttribute("client", bean);
```

```
/*建立饮料和配料的生产工厂，所有饮料和配料的实例均将通过它们获得。
```

这里工厂的实现还是针对具体实现，实际上也可以通过接口 Factory 建立起与具体工厂无关的实现。

```
*/
```

```
Beverage beverage = null ;
```

```
BeverageFactory factory = new BeverageFactory();
```

```
DecoratorFactory deco = new DecoratorFactory();
```

```
//从请求头中获得各个参数的值
```

```
product = request.getParameter("product").trim();
```

```
decorator = request.getParameter("decorator").trim();
```

```
num = request.getParameter("num").trim();
```

```
if(num != null && !num.equals(""))
```

```
    number = Integer.parseInt(num);
```

```
if(product != null && product != ""){
```

```
    //获得饮料对象
```

```
    beverage = factory.getBeverage(product);
```

```
    description = beverage.getDescription();
```

```
if(!(beverage instanceof NoBeverage) && !decorator.equals("") && number!=0){
```

```
    //获得配料对象
```

```
    beverage = deco.getDecorator(decorator, beverage);
```

```

        if(beverage instanceof NoDecorator)description = beverage.getDescription();
        else{
            //获得饮料被装饰后（即加入配料后）的描述
            description = beverage.getDescription()+" "+number+"份";

            for(int i=1;i<number;i++){
                //这个循环实现了添加多份配料
                beverage = deco.getDecorator(decorator, beverage);
            }
        }
    }
    //获得饮料被装饰后（即加入配料后）的价格
    price = beverage.getCost();

    //把所有参数设定到模型 bean 中去
    bean.setProduct(product);
    bean.setDecorator(decorator);
    bean.setDescription(description);
    bean.setPrice(price);

    //处理完毕后进行页面跳转，这是带参数形式的跳转，把请求头和响应对象返回给
    跳转的页面，
    //这样就可以使用 bean 里面的参数值
    RequestDispatcher dispatcher = request.getRequestDispatcher("index.jsp");
    dispatcher.forward(request, response);

}

public void doGet(HttpServletRequest request,HttpServletResponse response)
throws ServletException,IOException{
    doPost(request,response);
}
}

```

8.2.4 对非法请求的处理

用户的输入是不可预料的，我们最后提供一种对非法请求的处理方式。我们创建两个类 NoBeverage 和 NoDecorator，分别继承于 Beverage 类和 Decorator 类。工厂类也负责生

产这两个类，这样就可以统一对商品生产的访问。

(1) 不存在的饮料处理

```
/*
 *NoBeverage.java
 */
package lhl.hhu;
//添加这个装饰者类是为了使访问统一，处理各种非法请求
public class NoBeverage extends Beverage {
    String description = "没有您所点的饮料，请重新点(coca 或 coffee)。";
    public String getDescription(){
        return description;
    }
    public double getCost() {
        return 0;
    }
}
```

(2) 不存在的配料处理

```
/*
 *NoDecorator.java
 */
package lhl.hhu;
//添加这个装饰者类是为了使访问统一，处理各种非法请求
public class NoDecorator extends Decorator {
    String description = "没有您所点的配料，请重新点(milk 或 ice)。";
    public String getDescription() {
        return this.description;
    }
    public double getCost() {
        return 0;
    }
}
```

这样我们的功能就基本实现了，下面是我们整个项目的类图，如图 8-5 所示。

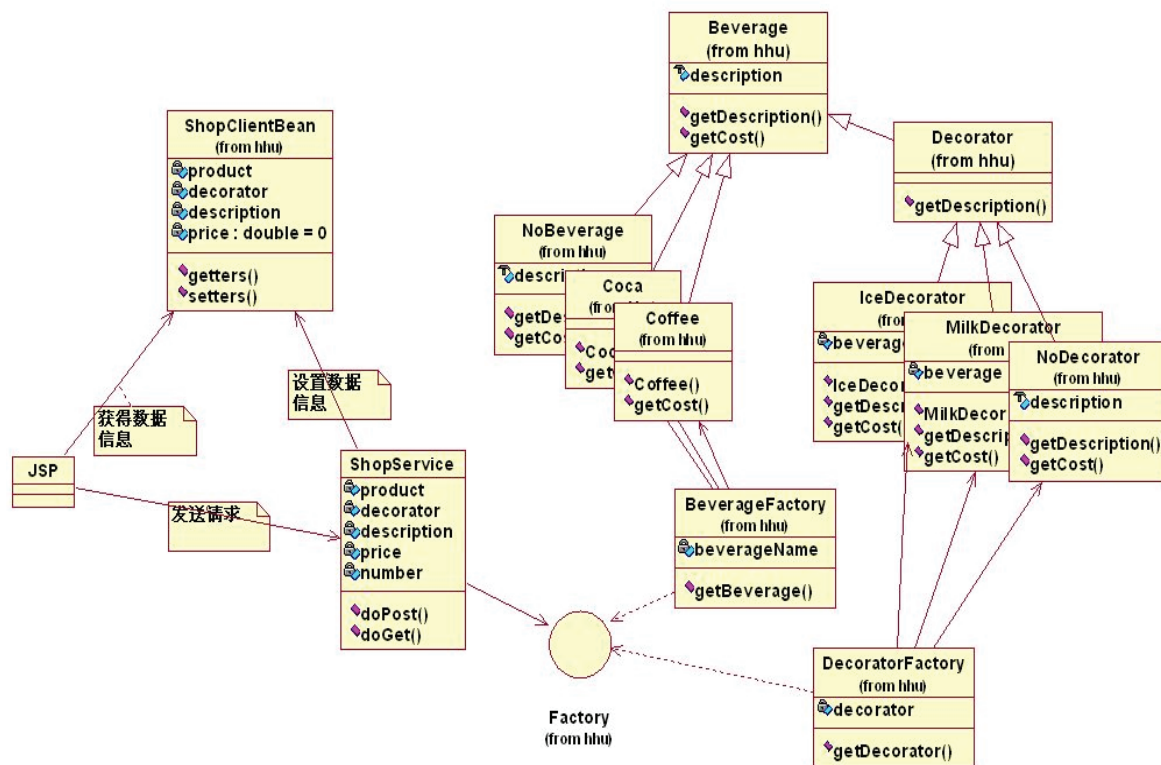


图 8-5 自动生产售货机程序的类图

8.3 开发环境的配置及效果

本章的例子在 myeclipse 平台上开发，以 Tomcat 作为 Web 服务器，在 Tomcat 上的文件组织如图 8-6 所示。

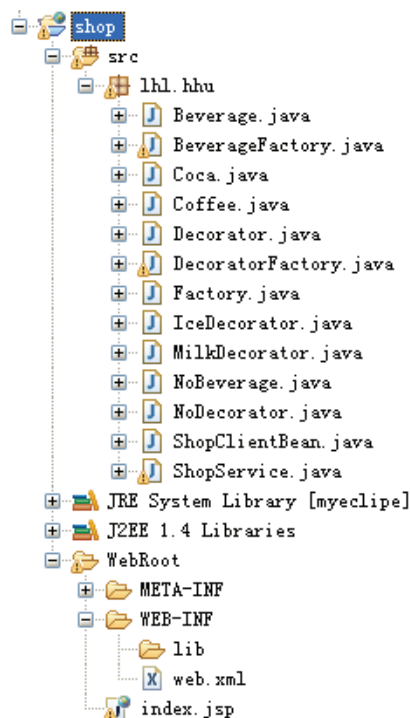


图 8-6 程序在 Tomcat 上的文件组织图

其中 web.xml 还需要具体配置，目的是为 JSP 提交请求后到 ShopService 类，如图 8-7 所示。

```
<?xml version="1.0" encoding="UTF-8" ?>
- <web-app version="2.4" xmlns="http://java.sun.com/xml/ns/j2ee" xmlns:xsi="http://www.w3.org/2001/XMLSchema-instance"
  xsi:schemaLocation="http://java.sun.com/xml/ns/j2ee http://java.sun.com/xml/ns/j2ee/web-app_2_4.xsd">
- <servlet>
  <servlet-name>shopService</servlet-name>
  <servlet-class>lhl.hhu.ShopService</servlet-class>
</servlet>
- <servlet-mapping>
  <servlet-name>shopService</servlet-name>
  <url-pattern>/shopService</url-pattern>
</servlet-mapping>
</web-app>
```

图 8-7 XML 配置文件

另外，myeclipse 和 Tomcat 配置好后可以直接将编译好的文件部署到 Tomcat 目录下，然后启动 Tomcat 后，从 IE 中输入一下网址：http://localhost:8080/shop/index.jsp 即可访问。

下面用一些截图展示效果：

1、进入页面后可看到如下界面，可以在输入框内输入需要提交的请求。如图 8-8，饮料为 coca，另外加上两份 milk 配料。在页面最下面显示当前用户提交的购买请求，以及相应的价格。

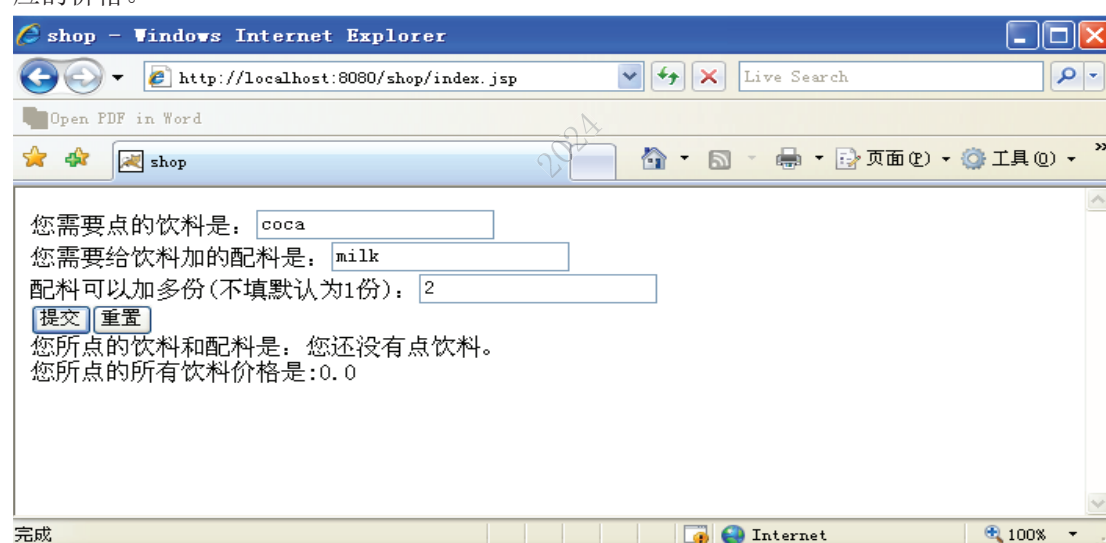


图 8-8 显示页面 1

点击“提交”按钮后得到下面界面：

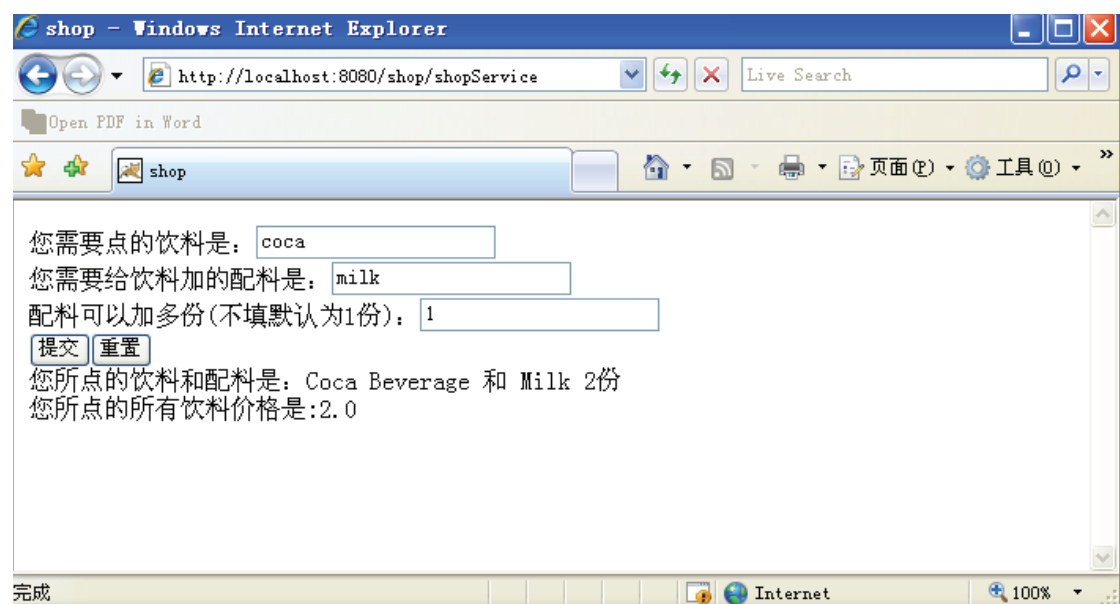


图 8-9 显示页面 2

2、如果输入了非法的购买请求（即商店里不存在相应的饮料和配料），则显示如下页面：

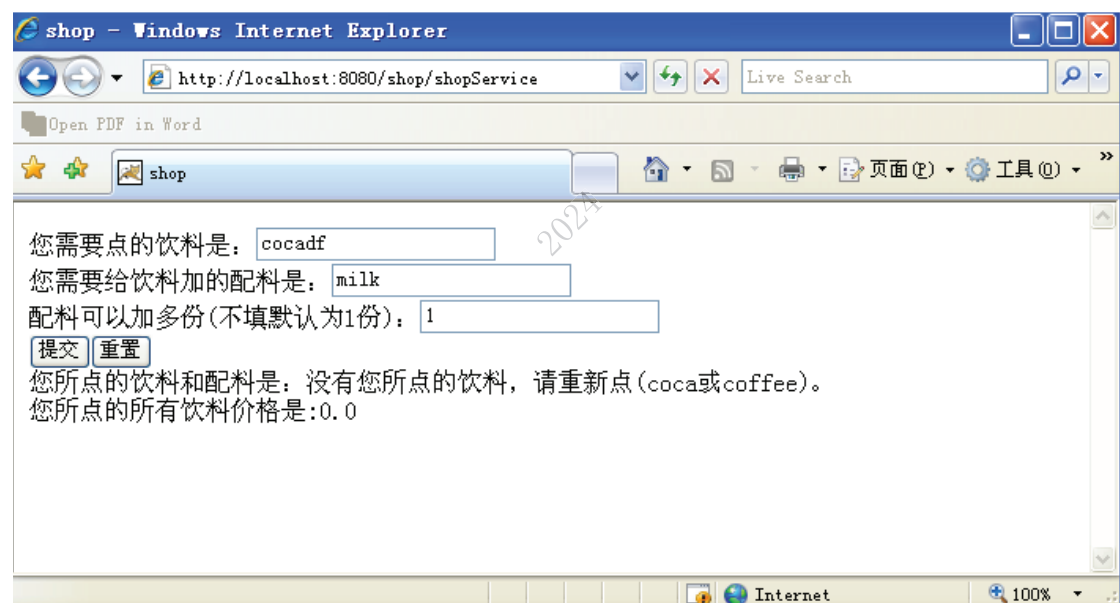


图 8-10 显示页面 3

参考文献

2024